An-Najah National University
Faculty of Engineering and Information Technology
Distributed and Operating Systems — Fall 2026

Bazar.com

Multi-tier Online Book Store

Prepared by: Lana Hawash & Zina Marhiv
Student IDs: 12217434 & 12217336

1. Project Overview

Bazar.com is a distributed online bookstore developed using a two-tier microservices architecture. The system consists of three independent services: Frontend, Catalog, and Order. These services communicate through HTTP REST APIs to support book searching, information retrieval, and purchasing.

The main objective is to demonstrate distributed communication, persistent data storage, concurrent request handling, and containerized deployment using Docker Compose.

Technologies Used

Technology	Purpose
Node.js	JavaScript runtime environment for the three services.
Express.js	Lightweight framework used to implement REST APIs.
TypeScript	Used to develop the services with type checking.
JSON Files	Store the book catalog and purchase history persistently.
Docker	Runs each microservice in a separate Linux container.
Docker Compose	Manages and connects the three containers.
HTTP REST APIs	Enable communication between the services using JSON responses.
PowerShell	Used to execute client requests and verify the results.
Git & GitHub	Used for version control, collaboration, and submission.

2. System Architecture

Bazar.com follows a two-tier architecture consisting of three independent microservices. The Frontend Service receives client requests and communicates with the Catalog and Order Services through HTTP REST APIs.

Each service runs in a separate Docker container, allowing the components to operate independently while communicating over the Docker network.

Microservices Overview

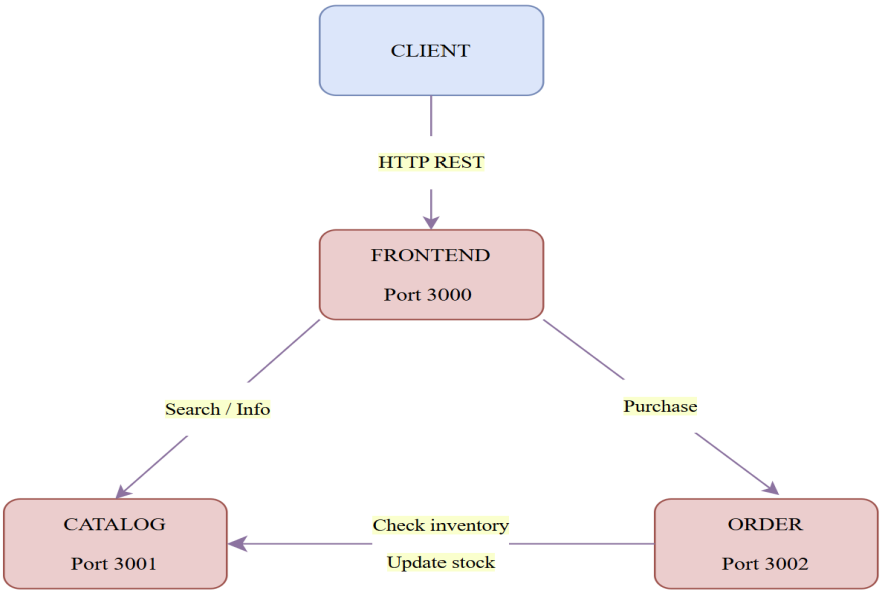
Service	Port	Responsibility

Frontend	3000	Receives client requests and forwards them to the appropriate backend service.
Catalog	3001	Manages book information, inventory, prices, and catalog updates.
Order	3002	Processes purchases, checks availability, updates stock, and records completed orders.

Service Communication

Search and information requests are forwarded from the Frontend to the Catalog Service. Purchase requests are forwarded to the Order Service, which communicates with the Catalog Service to verify availability and update inventory.

Figure 1: Bazar.com Microservices Architecture



3. Implementation and Workflow

The three microservices were implemented using TypeScript and Express.js. Each service provides REST endpoints and exchanges data in JSON format.

3.1 REST API Implementation

The following endpoints implement the main operations required by the bookstore.

Service	Method	Endpoint	Purpose

Frontend	GET	/search/:topic	Search books by topic.
Frontend	GET	/info/:id	Retrieve book details.
Frontend	POST	/purchase/:id	Forward purchase requests.
Catalog	GET	/search/:topic	Return matching books.
Catalog	GET	/info/:id	Return book information.
Catalog	PUT	/update/:id	Update stock or price.
Order	POST	/purchase/:id	Process and record purchases.

3.2 Purchase Workflow

When a client purchases a book, the request passes through the following steps:

1. The Frontend forwards the purchase request to the Order Service.
2. The Order Service queries the Catalog to verify availability.
3. The Catalog decreases the stock quantity by one.
4. The Order Service saves the completed purchase.
5. The Frontend returns the purchase confirmation to the client.

If the book is unavailable, the purchase is rejected without creating an order.

3.3 Data Persistence and Concurrency

The application uses two JSON files for persistent storage:

- catalog.json stores book information, prices, topics, and available quantities.
- orders.json stores completed purchases, including order IDs, book details, and timestamps.

Docker Compose mounts the data directories to preserve information when containers are removed and recreated.

To handle concurrent requests, the Catalog Service processes inventory updates sequentially using a Promise queue. This prevents simultaneous stock updates within the service process from overwriting each other or reducing inventory below zero.

The Order Service also uses a Promise queue to prevent overlapping order writes from losing records.

The complete project structure, including the source code and configuration files for each microservice, is available in the GitHub repository.

4. Docker Deployment

Each microservice runs in a separate Linux Docker container. Docker Compose is used to build, start, and connect the three services through an internal network.

The Frontend Service is accessible from the host at localhost:3000, while the Catalog and Order Services communicate internally using their Docker service names.

The compose.yaml file defines the service configurations, dependencies, and persistent data mounts.

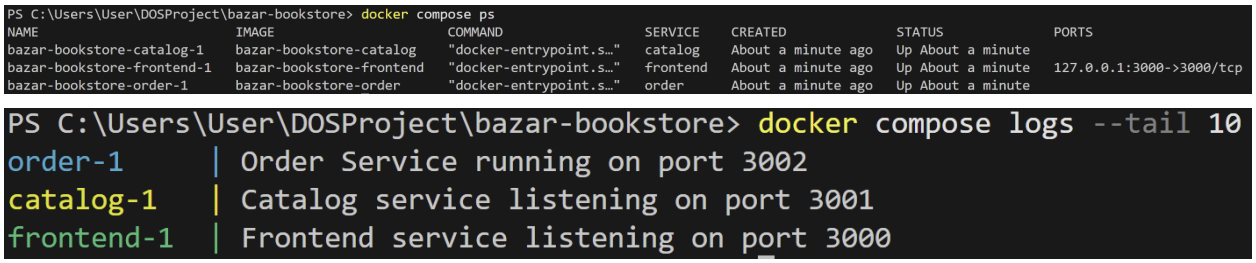


Figure 2. Docker Compose container status and startup logs confirming that all three microservices are running successfully.

5. Testing and Results

5.1 Search by Topic

A GET request was sent to /search/distributed%20systems through the Frontend Service. The Catalog Service successfully returned two matching books with their IDs and titles.

Figure 3. Search request showing the two books in the Distributed Systems category.

```
PS C:\Users\User\DOSProject\bazar-bookstore> Invoke-RestMethod "http://localhost:3000/search/distributed%20systems" | ConvertTo-Json -Depth 5
{
    "value":  [
                  {
                      "id":  1,
                      "title":  "How to get a good grade in DOS in 40 minutes a day."
                  },
                  {
                      "id":  2,
                      "title":  "RPCs for Noobs."
                  }
              ],
    "Count":  2
}
```

5.2 Book Information

A GET request to /info/2 successfully retrieved the details of *RPCs for Noobs*, including its topic, price (55), and available stock (9 copies).

Figure 4. Book information response showing the initial quantity before purchase.

```
PS C:\Users\User\DOSProject\bazar-bookstore> Invoke-RestMethod "http://localhost:3000/info/2" | ConvertTo-Json -Depth 5
{
    "id":  2,
    "title":  "RPCs for Noobs.",
    "topic":  "distributed systems",
    "quantity":  9,
    "price":  55
}
```

5.3 Purchase and Inventory Update

A POST request to /purchase/2 successfully purchased *RPCs for Noobs*, creating Order 11 and returning the required confirmation message.

The stock decreased from **9 to 8 copies**, and the completed purchase was saved in orders.json, confirming that both the inventory and order history were updated successfully.

Figure 5. Successful purchase response, updated inventory, and saved order record.

```
PS C:\Users\User\DOSProject\bazar-bookstore> Invoke-RestMethod -Method Post -Uri "http://localhost:3000/purchase/2" | ConvertTo-Json -Depth 5
{
    "message":  "bought book RPCs for Noobs.",
    "order":  {
                  "orderId":  11,
                  "bookId":  2,
                  "title":  "RPCs for Noobs.",
                  "price":  55,
                  "timestamp":  "2026-11-06T13:53:55.749Z"
              }
}
```

```
PS C:\Users\User\DOSProject\bazar-bookstore> Invoke-RestMethod "http://localhost:3000/info/2" | ConvertTo-Json -Depth 5
{
    "id":  2,
    "title":  "RPCs for Noobs.",
    "topic":  "distributed systems",
    "quantity":  8,
    "price":  55
}
```

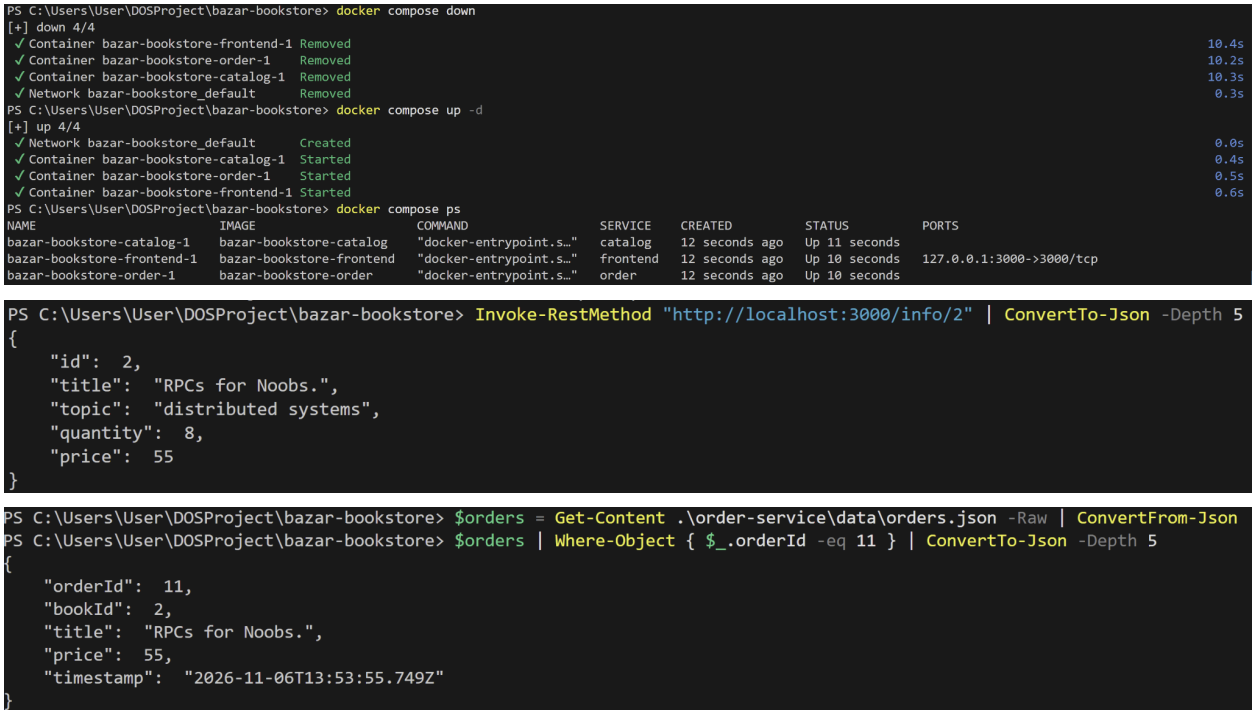
```
PS C:\Users\User\DOSProject\bazar-bookstore> Get-Content .\order-service\data\orders.json -Raw |
>>      ConvertFrom-Json |
>>      Select-Object -Last 1 |
>>      ConvertTo-Json -Depth 5
                    {
                        "orderId":  11,
                        "bookId":  2,
                        "title":  "RPCs for Noobs.",
                        "price":  55,
                        "timestamp":  "2026-11-06T13:53:55.749Z"
                    }
                ],
    "Count":  11
}
```

5.4 Persistence Verification

The three Docker containers were removed and recreated using docker compose down and docker compose up -d.

After restarting, Book 2 still had **8 copies**, and Order 11 remained stored in orders.json. This confirms that inventory updates and purchase records persist across container recreation.

Figure 6. Docker container recreation and verification of persistent inventory and order data.

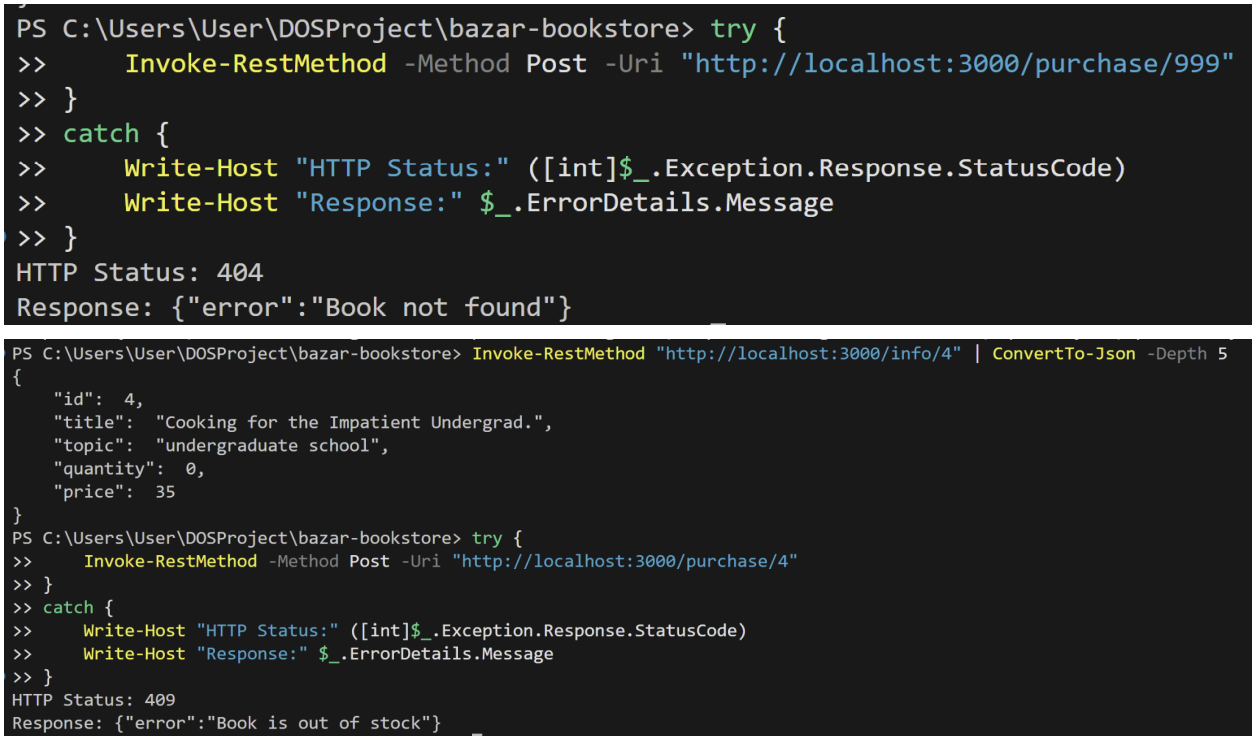


5.5 Error Handling

The application was tested with invalid purchase requests. Purchasing a nonexistent book (ID 999) returned **HTTP 404**, while purchasing an out-of-stock book (ID 4) returned **HTTP 409**.

Both requests were correctly rejected with clear JSON error messages.

Figure 7. Error-handling results for nonexistent and out-of-stock books.



5.6 Concurrency Testing

Five concurrent purchase requests were sent for Book 4, with only three copies available. The system successfully processed three purchases (HTTP 201) and rejected two requests (HTTP 409).

Three unique orders (12, 13, and 14) were recorded. The final inventory was checked to verify that no additional copies were sold.

Figure 8. Concurrent purchase results showing three successful orders and two rejected requests.



```
PS C:\Users\User\DOSProject\bazar-bookstore> $requests = 1..5 | ForEach-Object {
>>     Start-Job -ScriptBlock {
>>         try {
>>             $response = Invoke-WebRequest `
>>                 -Method Post `
>>                 -Uri "http://localhost:3000/purchase/4" `
>>                 -UseBasicParsing
>>
>>             [PSCustomObject]@{
>>                 Status = [int]$response.StatusCode
>>                 Result = "Success"
>>             }
>>         }
>>         catch {
>>             [PSCustomObject]@{
>>                 Status = [int]$_.Exception.Response.StatusCode
>>                 Result = $_.ErrorDetails.Message
>>             }
>>         }
>>     }
>> }
>>
>> $results = $requests | Wait-Job | Receive-Job
>> $requests | Remove-Job
>>
>> $results | Format-Table -AutoSize

Status Result                           RunspaceId
------ ------                           ----------
   201 Success                          ffe13e4c-f1f9-4cba-b215-34364648e448
   201 Success                          a073ca90-f9fe-4e69-b227-743b039c2578
   201 Success                          539f57a5-c585-4a39-a218-16664f5345d0
   409 {"error":"Book is out of stock"} 31a33ba2-4c76-4419-81be-afc7e452849c
   409 {"error":"Book is out of stock"} fa3eba1b-c986-4c5e-b757-84f7de63cdae
```

```
PS C:\Users\User\DOSProject\bazar-bookstore> Invoke-RestMethod "http://localhost:3000/info/4" |
>>     ConvertTo-Json -Depth 5
{
    "id":  4,
    "title":  "Cooking for the Impatient Undergrad.",
    "topic":  "undergraduate school",
    "quantity":  0,
    "price":  35
}
```

```
PS C:\Users\User\DOSProject\bazar-bookstore> $orders = Get-Content .\order-service\data\orders.json -Raw |
>>     ConvertFrom-Json
>>
>> $orders |
>>     Select-Object -Last 3 |
>>     Format-Table orderId, bookId, title, price, timestamp -AutoSize

orderId bookId title                              price timestamp
------- ------ -----                              ----- ---------
     12      4 Cooking for the Impatient Undergrad.    35 2026-11-06T14:31:19.472Z
     13      4 Cooking for the Impatient Undergrad.    35 2026-11-06T14:31:19.547Z
     14      4 Cooking for the Impatient Undergrad.    35 2026-11-06T14:31:19.789Z
```

6. Design Decisions, Limitations, and Improvements

6.1 Design Decisions and Tradeoffs

- **Node.js, Express, and TypeScript:** Selected to build lightweight REST microservices with asynchronous request handling and type checking.
- **JSON-based storage:** Chosen for simplicity and easy data inspection without requiring a database server. However, reading and rewriting complete files is less efficient for larger systems.
- **Docker Compose:** Used to deploy and connect the three independent services with a single configuration file.
- **Promise queues:** Used to process inventory updates and order writes sequentially, preventing conflicting modifications within each service process.

6.2 Known Limitations

Although the application works correctly in the tested scenarios, inventory updates and order recording are separate operations. A service failure between these operations could cause inconsistent data.

The Promise queues protect concurrent writes within a single service process but do not support multiple instances sharing the same JSON files.

6.3 Possible Improvements

Future improvements could include using SQLite transactions for stronger data consistency, adding unique purchase request identifiers to prevent duplicate orders, and implementing automated recovery for interrupted purchases.

7. How to Run the Project

The application requires Docker Desktop with Linux containers enabled.

From the repository's root directory, execute:

```
docker compose up --build -d
docker compose ps
```

The Frontend Service becomes accessible at

```
http://localhost:3000
```

The main REST operations can be tested using PowerShell:

```
Invoke-RestMethod "http://localhost:3000/search/distributed%20systems"
Invoke-RestMethod "http://localhost:3000/info/2"
Invoke-RestMethod -Method Post -Uri "http://localhost:3000/purchase/2"
```

To view service logs or stop the application:

```
docker compose logs --tail 10
docker compose down
```

8. Conclusion

Bazar.com successfully implements a distributed bookstore using three independent microservices communicating through REST APIs. The application supports book searches, information retrieval, and purchases while maintaining persistent inventory and order records.

Testing demonstrated successful Docker deployment, correct stock updates, data persistence, error handling, and concurrent purchase processing.